Guide d'exécution – Projet BikeX (V1)

Symfony 7 · PHP 8.3.12 · MySQL 8.0.30 · Apache · Twig Atomic Design

> 📋 **Objectif de ce document**
> Guide étape par étape pour transformer l'intégration HTML/CSS de BikeX en une application Symfony 7 fonctionnelle.
> Chaque étape est concrète, avec les commandes exactes et les fichiers à créer.

Version: 1.0
Date: 10 février 2026
Projet: BikeX – Plateforme E-Commerce Vélos Haut de Gamme
Stack: Symfony 7 (--webapp) · PHP 8.3.12 · MySQL 8.0.30 · Apache · Twig (Atomic Design) · Fixtures

Table des matières

1. Prérequis et environnement
2. Installation du projet Symfony
3. Architecture des dossiers
4. Twig et Atomic Design
5. Base de données et entités Doctrine
6. Fixtures (données de test)
7. Contrôleurs et routes
8. Intégration des maquettes HTML/CSS
9. Formulaires Symfony
10. Sécurité et authentification
11. Services et logique métier
12. Tests
13. Déploiement et configuration Apache
14. Checklist de validation V1

1. Prérequis et environnement

1.1 Logiciels requis

Logiciel	Version exacte	Vérification
PHP	8.3.12	`php -v`
Composer	2.x (dernière stable)	`composer --version`
MySQL	8.0.30	`mysql --version`
Apache	2.4.x	`httpd -v` ou via Laragon
Node.js	18+ (pour Webpack Encore)	`node -v`
npm ou **yarn**	Dernière stable	`npm -v` / `yarn -v`
Symfony CLI (recommandé)	Dernière stable	`symfony version`
Git	2.x	`git --version`

1.2 Extensions PHP requises

Vérifiez avec `php -m` que ces extensions sont activées :

```
☑ pdo_mysql
☑ intl
```

```
☑ mbstring
☑ xml
☑ curl
☑ zip
☑ gd (pour manipulation d'images)
☑ opcache (recommandé en production)
```

> 💡 **Avec Laragon** : La plupart de ces extensions sont activées par défaut. Sinon, décommentez-les dans `php.ini`.

1.3 Vérification de compatibilité Symfony

```
symfony check:requirements
```

1.4 Configuration de l'éditeur (VS Code)

Extensions recommandées :

- **PHP Intelephense** – Autocomplétion PHP
- **Twig Language 2** – Coloration syntaxique Twig
- **Symfony for VSCode** – Navigation Symfony
- **MySQL** – Client base de données intégré

---

2. Installation du projet Symfony

2.1 Création du projet

```
# Créer le projet Symfony avec la recette --webapp (inclut Twig, Doctrine, Security, Mailer, etc.)
symfony new bikex --webapp

# OU avec Composer si Symfony CLI n'est pas installé
composer create-project symfony/skeleton bikex
cd bikex
composer require webapp
```

> 💡 **La recette `--webapp`** installe automatiquement :
>
> - `twig` (moteur de templates)
> - `doctrine` (ORM + migrations)
> - `security-bundle` (authentification)
> - `form` (formulaires)
> - `validator` (validation)
> - `mailer` (emails)
> - `asset-mapper` ou `webpack-encore` (assets)
> - `debug-bundle` et `profiler` (barre de debug)

2.2 Configuration de la base de données

Éditez le fichier `.env` (ou mieux, `.env.local` qui n'est pas versionné) :

```
# .env.local
DATABASE_URL="mysql://root:@127.0.0.1:3306/bikex?serverVersion=8.0.30&charset=utf8mb4"
```

> ⚠ **Attention** : Adaptez `root:` avec votre utilisateur et mot de passe MySQL. Laragon utilise généralement `root` sans mot de passe.

2.3 Création de la base de données

```
php bin/console doctrine:database:create
```

2.4 Installation de Webpack Encore (pour les assets)

```
composer require symfony/webpack-encore-bundle
npm install
```

2.5 Initialisation Git

```
git init
git add .
git commit -m "feat: initialisation projet Symfony 7 BikeX"
```

2.6 Lancement du serveur de développement

```
# Avec Symfony CLI (recommandé)
symfony server:start

# OU avec le serveur PHP intégré
php -S localhost:8000 -t public/
```

> 💡 **Avec Apache/Laragon** : Configurez un VirtualHost pointant vers le dossier `public/` du projet (voir section 13).

---

3. Architecture des dossiers

3.1 Structure Symfony standard (après `--webapp`)

```
bikex/
├── assets/                    ← Assets front-end (JS, CSS, images)
│   ├── controllers/
│   ├── styles/
│   │   └── app.css            ← Styles globaux (importer styles.css ici)
│   └── app.js
│
├── bin/
│   └── console                ← CLI Symfony
│
├── config/                    ← Configuration de l'application
│   ├── packages/              ← Config par bundle (doctrine.yaml, security.yaml...)
│   ├── routes/
│   └── services.yaml
│
├── migrations/                ← Migrations Doctrine (historique BDD)
│
├── public/                    ← Document root Apache (seul dossier exposé)
│   ├── index.php              ← Front controller
│   └── build/                 ← Assets compilés par Webpack Encore
│
├── src/                       ← Code source PHP
│   ├── Controller/            ← Contrôleurs
│   ├── Entity/                ← Entités Doctrine (modèles)
│   ├── Form/                  ← Formulaires Symfony
│   ├── Repository/            ← Repositories Doctrine
│   ├── Service/               ← Services métier
```

```
│   ├── DataFixtures/          ← Fixtures (données de test)
│   └── Security/              ← Authentification
│
├── templates/                 ← Templates Twig (voir section Atomic Design)
│   ├── base.html.twig         ← Layout principal
│   ├── components/            ← Atomic Design (atoms, molecules, organisms)
│   └── pages/                 ← Pages complètes
│
├── tests/                     ← Tests PHPUnit
│
├── .env                       ← Variables d'environnement (commit)
├── .env.local                 ← Variables locales (pas commit)
├── composer.json
├── package.json
└── webpack.config.js
```

3.2 Structure Twig – Atomic Design (détaillé en section 4)

```
templates/
├── base.html.twig
│
├── components/
│   ├── atoms/                 ← Éléments indivisibles
│   │   ├── button.html.twig
│   │   ├── input.html.twig
│   │   ├── label.html.twig
│   │   ├── icon.html.twig
│   │   ├── image.html.twig
│   │   ├── badge.html.twig
│   │   └── price.html.twig
│   │
│   ├── molecules/             ← Groupes d'atomes
│   │   ├── nav-link.html.twig
│   │   ├── search-bar.html.twig
│   │   ├── product-card.html.twig
│   │   ├── cart-item.html.twig
│   │   ├── form-group.html.twig
│   │   ├── step-indicator.html.twig
│   │   └── order-summary-item.html.twig
│   │
│   └── organisms/             ← Groupes de molécules
│       ├── navbar.html.twig
│       ├── hero-section.html.twig
│       ├── product-grid.html.twig
│       ├── sidebar-filters.html.twig
│       ├── sidebar-cart-summary.html.twig
│       ├── checkout-form.html.twig
│       └── footer.html.twig
│
├── pages/                     ← Templates de pages (= les pages HTML actuelles)
│   ├── home/
│   │   └── index.html.twig
│   ├── catalog/
│   │   ├── category.html.twig
│   │   └── product_detail.html.twig
│   ├── cart/
│   │   └── index.html.twig
│   ├── checkout/
│   │   ├── information.html.twig
│   │   ├── payment.html.twig
│   │   └── confirmation.html.twig
│   └── security/
│       ├── login.html.twig
│       └── register.html.twig
```

```
│
└── emails/                    ← Templates d'emails transactionnels
    ├── order_confirmation.html.twig
    └── welcome.html.twig
```

---

4. Twig et Atomic Design

4.1 Principe de l'Atomic Design

L'Atomic Design (Brad Frost) décompose l'interface en 5 niveaux :

```
ATOMES → MOLÉCULES → ORGANISMES → TEMPLATES → PAGES

Atome      = élément indivisible (bouton, input, icône, image)
Molécule   = groupe d'atomes (carte produit, champ de formulaire avec label)
Organisme  = groupe de molécules (navbar, grille produits, footer)
Template   = squelette de page (layout avec zones vides)
Page       = template rempli avec des données réelles
```

4.2 Le layout de base (`base.html.twig`)

Ce fichier remplace le HTML répétitif (head, navbar, footer) présent dans chaque page HTML actuelle :

```
{# templates/base.html.twig #}
<!DOCTYPE html>
<html lang="fr">
<head>
    <meta charset="UTF-8">
    <title>{% block title %}BikeX{% endblock %}</title>
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <meta name="description" content="{% block meta_description %}BikeX - Vélos haut de gamme{% endblock
%}">

    {# Bootstrap 5 CDN #}
    <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css"
rel="stylesheet">

    {# Bootstrap Icons #}
    <link rel="stylesheet" href="https://cdn.jsdelivr.net/npm/bootstrap-icons@1.11.0/font/bootstrap-
icons.css">

    {# Webpack Encore – fichiers compilés #}
    {% block stylesheets %}
        {{ encore_entry_link_tags('app') }}
    {% endblock %}
</head>
<body>
    {# ─── NAVBAR (organisme) ─── #}
    {% include 'components/organisms/navbar.html.twig' %}

    {# ─── CONTENU PRINCIPAL ─── #}
    <main class="container my-4">
        {# Messages flash (succès, erreur) #}
        {% for label, messages in app.flashes %}
            {% for message in messages %}
                <div class="alert alert-{{ label }} alert-dismissible fade show" role="alert">
                    {{ message }}
                    <button type="button" class="btn-close" data-bs-dismiss="alert"></button>
                </div>
            {% endfor %}
        {% endfor %}
```

```
        {% block body %}{% endblock %}
    </main>

    {# ─── FOOTER (organisme) ─── #}
    {% include 'components/organisms/footer.html.twig' %}

    {# Bootstrap JS #}
    <script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap.bundle.min.js"></script>

    {% block javascripts %}
        {{ encore_entry_script_tags('app') }}
    {% endblock %}
</body>
</html>
```

4.3 Exemples d'atomes

Bouton (`atoms/button.html.twig`)

```
{# templates/components/atoms/button.html.twig #}
{#
    Paramètres :
    - label    : texte du bouton (obligatoire)
    - type     : 'primary' | 'outline-secondary' | 'outline-danger' (défaut: 'primary')
    - size     : 'sm' | '' | 'lg' (défaut: '')
    - href     : si défini, rend un lien <a> au lieu d'un <button>
    - fullWidth: true/false (défaut: false)
    - htmlType : 'button' | 'submit' (défaut: 'button')
#}
{% set btnClass = 'btn btn-' ~ (type|default('primary')) %}
{% if size|default('') %}{% set btnClass = btnClass ~ ' btn-' ~ size %}{% endif %}
{% if fullWidth|default(false) %}{% set btnClass = btnClass ~ ' w-100' %}{% endif %}

{% if href is defined and href %}
    <a href="{{ href }}" class="{{ btnClass }}">{{ label }}</a>
{% else %}
    <button type="{{ htmlType|default('button') }}" class="{{ btnClass }}">{{ label }}</button>
{% endif %}
```

Prix (`atoms/price.html.twig`)

```
{# templates/components/atoms/price.html.twig #}
{#
    Paramètres :
    - amount : le montant (number)
    - class  : classes CSS supplémentaires
#}
<span class="product-price {{ class|default('') }}">
    {{ amount|number_format(0, ',', ' ') }} €
</span>
```

4.4 Exemples de molécules

Carte produit (`molecules/product-card.html.twig`)

```
{# templates/components/molecules/product-card.html.twig #}
{#
```

```
    Paramètres :
    - product : objet Product (entité Doctrine)
#}
<div class="product-card">
    <a href="{{ path('app_product_detail', {slug: product.slug}) }}" class="text-decoration-none text-
dark">
        {% include 'components/atoms/image.html.twig' with {
            src: asset('uploads/products/' ~ product.imagePath),
            alt: product.nom
        } %}
        <div class="product-name">{{ product.nom }}</div>
        {% include 'components/atoms/price.html.twig' with { amount: product.prix } %}
    </a>
</div>
```

Article panier (`molecules/cart-item.html.twig`)

```
{# templates/components/molecules/cart-item.html.twig #}
<tr>
    <td>
        <div class="d-flex align-items-center">
            <img src="{{ asset('uploads/products/' ~ item.product.imagePath) }}"
                 alt="{{ item.product.nom }}" class="me-3" style="width:80px;">
            <div>
                <div class="fw-semibold">{{ item.product.nom }}</div>
                <small class="text-muted">Taille {{ item.taille }}</small>
            </div>
        </div>
    </td>
    <td>{% include 'components/atoms/price.html.twig' with { amount: item.product.prix } %}</td>
    <td style="width:120px;">
        <input type="number" class="form-control" name="quantities[{{ item.id }}]"
               value="{{ item.quantite }}" min="1">
    </td>
    <td>{% include 'components/atoms/price.html.twig' with { amount: item.sousTotal } %}</td>
    <td>
        {% include 'components/atoms/button.html.twig' with {
            label: 'Supprimer', type: 'outline-danger', size: 'sm'
        } %}
    </td>
</tr>
```

4.5 Exemples d'organismes

Navbar (`organisms/navbar.html.twig`)

```
{# templates/components/organisms/navbar.html.twig #}
<nav class="navbar navbar-expand-lg navbar-bikex">
    <div class="container">
        <a class="navbar-brand" href="{{ path('app_home') }}">
            <img src="{{ asset('build/images/bikex-logo.png') }}" alt="BikeX">
        </a>
        <button class="navbar-toggler text-white" type="button" data-bs-toggle="collapse"
                data-bs-target="#navbarNav">
            <span class="navbar-toggler-icon"></span>
        </button>
        <div class="collapse navbar-collapse" id="navbarNav">
            <ul class="navbar-nav me-auto mb-2 mb-lg-0">
                <li class="nav-item">
                    <a class="nav-link {{ app.request.attributes.get('_route') starts with 'app_home' ?
'active' }}"
```

```
                        href="{{ path('app_home') }}">Vélos</a>
                </li>
                <li class="nav-item"><a class="nav-link" href="#">Pièces</a></li>
                <li class="nav-item"><a class="nav-link" href="#">Accessoires</a></li>
                <li class="nav-item"><a class="nav-link" href="#">Marques</a></li>
                <li class="nav-item"><a class="nav-link" href="#">Contact</a></li>
            </ul>
            <ul class="navbar-nav ms-auto">
                <li class="nav-item me-3">
                    <a class="nav-link" href="#"><i class="bi bi-search"></i></a>
                </li>
                <li class="nav-item me-3">
                    {% if app.user %}
                        <a class="nav-link" href="{{ path('app_profile') }}">
                            <i class="bi bi-person-fill"></i>
                        </a>
                    {% else %}
                        <a class="nav-link" href="{{ path('app_login') }}">
                            <i class="bi bi-person"></i>
                        </a>
                    {% endif %}
                </li>
                <li class="nav-item">
                    <a class="nav-link" href="{{ path('app_cart') }}">
                        <i class="bi bi-cart"></i>
                        {% if cart_count|default(0) > 0 %}
                            ({{ cart_count }})
                        {% endif %}
                    </a>
                </li>
            </ul>
        </div>
    </div>
</nav>
```

Footer (`organisms/footer.html.twig`)

```
{# templates/components/organisms/footer.html.twig #}
<footer class="container">
    <div class="d-flex justify-content-between">
        <span>&copy; {{ "now"|date("Y") }} BikeX</span>
        <span>
            <a href="#" class="text-decoration-none text-muted">Conditions générales</a> ·
            <a href="#" class="text-decoration-none text-muted">Mentions légales</a>
        </span>
    </div>
</footer>
```

4.6 Exemple de page complète

Page d'accueil (`pages/home/index.html.twig`)

```
{# templates/pages/home/index.html.twig #}
{% extends 'base.html.twig' %}

{% block title %}BikeX - Vélos Haut de Gamme{% endblock %}

{% block body %}
    {# Hero Section #}
    {% include 'components/organisms/hero-section.html.twig' with {
        title: 'Nouvelle collection<br>Vélos de route',
```

```
        subtitle: 'Découvrez nos modèles carbone légers et performants.',
        cta_label: 'Découvrir',
        cta_href: path('app_category', {slug: 'route'}),
        image: asset('build/images/hero-bike.jpg')
    } %}

    {# Meilleures ventes #}
    <section class="mt-4">
        <h2 class="h5 mb-3">Meilleures ventes</h2>

        {# Filtres de catégories #}
        <div class="mb-3">
            {% for category in categories %}
                {% include 'components/atoms/button.html.twig' with {
                    label: category.nom,
                    type: 'outline-secondary',
                    size: 'sm'
                } %}
            {% endfor %}
        </div>

        {# Grille de produits #}
        {% include 'components/organisms/product-grid.html.twig' with {
            products: bestSellers,
            columns: 4
        } %}
    </section>
{% endblock %}
```

---

5. Base de données et entités Doctrine

5.1 Créer les entités avec MakerBundle

> 💡 **Le MakerBundle** génère le code pour vous. C'est le moyen le plus rapide et le plus fiable.

```
# Entité Categorie
php bin/console make:entity Categorie

# Entité Produit
php bin/console make:entity Produit

# Entité Utilisateur (spécial : utiliser make:user)
php bin/console make:user Utilisateur

# Entité Commande
php bin/console make:entity Commande

# Entité ArticleCommande (ligne de commande)
php bin/console make:entity ArticleCommande

# Entité Adresse
php bin/console make:entity Adresse
```

5.2 Entité `Categorie`

Champs à créer lors du `make:entity` :

Champ	Type	Options
`nom`	string (255)	not null
`slug`	string (255)	unique

Champ	Type	Options
`description`	text	nullable
`image`	string (255)	nullable

Relation : `OneToMany` vers `Produit`

5.3 Entité `Produit`

Champ	Type	Options
`nom`	string (255)	not null
`slug`	string (255)	unique
`description`	text	not null
`prix`	decimal (10,2)	not null
`poids`	float	nullable
`dimensions`	string (100)	nullable
`materiauCadre`	string (100)	nullable
`typeCadre`	string (50)	nullable
`typeSuspension`	string (50)	nullable
`imagePath`	string (255)	nullable
`disponibilite`	integer	default 0
`categorie`	ManyToOne → Categorie	not null
`dateAjout`	datetime_immutable	auto
`dateModification`	datetime_immutable	auto

5.4 Entité `Utilisateur` (via `make:user`)

Champ	Type	Options
`email`	string (180)	unique, identifiant
`password`	string (255)	hashed
`nom`	string (100)	not null
`prenom`	string (100)	not null
`telephone`	string (20)	nullable
`roles`	json	default ['ROLE_USER']
`dateInscription`	datetime_immutable	auto

Relations : `OneToMany` vers `Commande`, `OneToMany` vers `Adresse`

5.5 Entité `Commande`

Champ	Type	Options
`reference`	string (50)	unique
`statut`	string (20)	default 'en_attente'
`totalHt`	decimal (10,2)	not null
`totalTtc`	decimal (10,2)	not null

Champ	Type	Options
`modePaiement`	string (50)	nullable
`stripePaymentId`	string (255)	nullable
`utilisateur`	ManyToOne → Utilisateur	not null
`adresseLivraison`	ManyToOne → Adresse	not null
`adresseFacturation`	ManyToOne → Adresse	nullable
`dateCreation`	datetime_immutable	auto
`dateModification`	datetime_immutable	auto

Relation : `OneToMany` vers `ArticleCommande`

5.6 Entité `ArticleCommande`

Champ	Type	Options
`quantite`	integer	not null, min 1
`prixUnitaire`	decimal (10,2)	not null
`taille`	string (10)	nullable
`options`	json	nullable
`commande`	ManyToOne → Commande	not null
`produit`	ManyToOne → Produit	not null

5.7 Entité `Adresse`

Champ	Type	Options
`prenom`	string (100)	not null
`nom`	string (100)	not null
`rue`	string (255)	not null
`codePostal`	string (10)	not null
`ville`	string (100)	not null
`pays`	string (2)	default 'FR'
`utilisateur`	ManyToOne → Utilisateur	not null

5.8 Migrations

```
# Générer la migration
php bin/console make:migration

# Vérifier le SQL généré dans migrations/VersionXXXX.php
# Puis exécuter
php bin/console doctrine:migrations:migrate
```

> ⚠ **Attention** : Relisez TOUJOURS le fichier de migration avant de l'exécuter. Vérifiez les types, les contraintes et les index.

5.9 Schéma des relations

```
┌──────────────┐     ┌──────────────┐     ┌──────────────────┐
│  CATEGORIE   │──────<│   PRODUIT    │>──────│ ARTICLE_COMMANDE │
│              │ 1:N │              │ 1:N │                  │
└──────────────┘     └──────────────┘     └──────────┬───────┘
                                                     │ N:1
                                                     │
                                                     ┌──────────┴──────┐
┌──────────────┐     ┌──────────────┐     ┌──────────────────┐
│   ADRESSE    │>──────│ UTILISATEUR  │──────<│    COMMANDE      │
│              │ N:1 │              │ 1:N │                  │
└──────────────┘     └──────────────┘     └──────────────────┘
```

---

6. Fixtures (données de test)

6.1 Installation

```
composer require --dev orm-fixtures
composer require --dev fakerphp/faker
```

6.2 Structure recommandée des fixtures

```
src/DataFixtures/
├── AppFixtures.php           ← Point d'entrée (charge tout dans l'ordre)
├── CategorieFixtures.php     ← Catégories de vélos
├── ProduitFixtures.php       ← Produits avec données réalistes
├── UtilisateurFixtures.php   ← Utilisateurs de test
├── AdresseFixtures.php       ← Adresses de test
└── CommandeFixtures.php      ← Commandes de test
```

6.3 Exemple : `CategorieFixtures.php`

```php
<?php
// src/DataFixtures/CategorieFixtures.php

namespace App\DataFixtures;

use App\Entity\Categorie;
use Doctrine\Bundle\FixturesBundle\Fixture;
use Doctrine\Persistence\ObjectManager;

class CategorieFixtures extends Fixture
{
    public const CATEGORIES = [
        ['nom' => 'Vélos de route',       'slug' => 'route'],
        ['nom' => 'VTT',                  'slug' => 'vtt'],
        ['nom' => 'VTC / Urbain',         'slug' => 'urbain'],
        ['nom' => 'Gravel',               'slug' => 'gravel'],
        ['nom' => 'Vélos électriques',    'slug' => 'electrique'],
        ['nom' => 'Cargo',                'slug' => 'cargo'],
        ['nom' => 'Tandem',               'slug' => 'tandem'],
    ];

    public function load(ObjectManager $manager): void
    {
        foreach (self::CATEGORIES as $data) {
            $categorie = new Categorie();
            $categorie->setNom($data['nom']);
            $categorie->setSlug($data['slug']);
```

```
            $manager->persist($categorie);

            // Référence pour ProduitFixtures
            $this->addReference('categorie_' . $data['slug'], $categorie);
        }

        $manager->flush();
    }
}
```

6.4 Exemple : `ProduitFixtures.php`

```
<?php
// src/DataFixtures/ProduitFixtures.php

namespace App\DataFixtures;

use App\Entity\Produit;
use Doctrine\Bundle\FixturesBundle\Fixture;
use Doctrine\Common\DataFixtures\DependentFixtureInterface;
use Doctrine\Persistence\ObjectManager;

class ProduitFixtures extends Fixture implements DependentFixtureInterface
{
    public function load(ObjectManager $manager): void
    {
        $produits = [
            [
                'nom'            => 'Vélo de Route Carbone X',
                'slug'           => 'velo-route-carbone-x',
                'description'    => 'Vélo de route en carbone ultra léger, transmission 12 vitesses,
freins à disque hydrauliques.',
                'prix'           => 2990.00,
                'poids'          => 7.2,
                'materiauCadre'  => 'Carbone',
                'typeCadre'      => 'Course',
                'disponibilite'  => 15,
                'categorie'      => 'route',
                'imagePath'      => 'bike-placeholder.jpg',
            ],
            [
                'nom'            => 'Vélo de Route Endurance',
                'slug'           => 'velo-route-endurance',
                'description'    => 'Conçu pour les longues distances, confort optimal et performance.',
                'prix'           => 2490.00,
                'poids'          => 8.1,
                'materiauCadre'  => 'Carbone',
                'typeCadre'      => 'Endurance',
                'disponibilite'  => 20,
                'categorie'      => 'route',
                'imagePath'      => 'bike-placeholder.jpg',
            ],
            [
                'nom'            => 'Vélo de Route Ultralight',
                'slug'           => 'velo-route-ultralight',
                'description'    => 'Le plus léger de notre gamme, pour les montées les plus
exigeantes.',
                'prix'           => 3200.00,
                'poids'          => 6.5,
                'materiauCadre'  => 'Carbone',
                'typeCadre'      => 'Course',
                'disponibilite'  => 8,
                'categorie'      => 'route',
```

```
                'imagePath'      => 'bike-placeholder.jpg',
            ],
            [
                'nom'            => 'Vélo Aero Performance',
                'slug'           => 'velo-aero-performance',
                'description'    => 'Aérodynamisme poussé pour la vitesse en peloton et contre-la-
montre.',
                'prix'           => 3990.00,
                'poids'          => 7.8,
                'materiauCadre'  => 'Carbone',
                'typeCadre'      => 'Aero',
                'disponibilite'  => 5,
                'categorie'      => 'route',
                'imagePath'      => 'bike-placeholder.jpg',
            ],
            // Ajoutez d'autres produits pour les autres catégories...
        ];

        foreach ($produits as $data) {
            $produit = new Produit();
            $produit->setNom($data['nom']);
            $produit->setSlug($data['slug']);
            $produit->setDescription($data['description']);
            $produit->setPrix($data['prix']);
            $produit->setPoids($data['poids']);
            $produit->setMateriauCadre($data['materiauCadre']);
            $produit->setTypeCadre($data['typeCadre']);
            $produit->setDisponibilite($data['disponibilite']);
            $produit->setImagePath($data['imagePath']);
            $produit->setCategorie(
                $this->getReference('categorie_' . $data['categorie'], \App\Entity\Categorie::class)
            );

            $manager->persist($produit);
        }

        $manager->flush();
    }

    public function getDependencies(): array
    {
        return [CategorieFixtures::class];
    }
}
```

6.5 Exemple : `UtilisateurFixtures.php`

```
<?php
// src/DataFixtures/UtilisateurFixtures.php

namespace App\DataFixtures;

use App\Entity\Utilisateur;
use Doctrine\Bundle\FixturesBundle\Fixture;
use Doctrine\Persistence\ObjectManager;
use Symfony\Component\PasswordHasher\Hasher\UserPasswordHasherInterface;

class UtilisateurFixtures extends Fixture
{
    public function __construct(
        private UserPasswordHasherInterface $hasher
    ) {}

    public function load(ObjectManager $manager): void
```

```
    {
        // Admin
        $admin = new Utilisateur();
        $admin->setEmail('admin@bikex.com');
        $admin->setNom('AQUASAPONE');
        $admin->setPrenom('Marco');
        $admin->setRoles(['ROLE_ADMIN']);
        $admin->setPassword($this->hasher->hashPassword($admin, 'Admin123!'));
        $manager->persist($admin);
        $this->addReference('user_admin', $admin);

        // Client test
        $client = new Utilisateur();
        $client->setEmail('jean.dupont@example.com');
        $client->setNom('Dupont');
        $client->setPrenom('Jean');
        $client->setRoles(['ROLE_USER']);
        $client->setPassword($this->hasher->hashPassword($client, 'Client123!'));
        $manager->persist($client);
        $this->addReference('user_client', $client);

        $manager->flush();
    }
}
```

6.6 Chargement des fixtures

```
# Charger toutes les fixtures (ATTENTION : vide la base !)
php bin/console doctrine:fixtures:load

# Charger sans confirmation
php bin/console doctrine:fixtures:load --no-interaction

# Ajouter sans vider (append)
php bin/console doctrine:fixtures:load --append
```

> ⚠ **Attention** : `doctrine:fixtures:load` SANS `--append` supprime toutes les données existantes.

---

7. Contrôleurs et routes

7.1 Liste des contrôleurs à créer

```
php bin/console make:controller HomeController
php bin/console make:controller CatalogController
php bin/console make:controller ProductController
php bin/console make:controller CartController
php bin/console make:controller CheckoutController
php bin/console make:controller OrderController
```

7.2 Table des routes (V1)

Route	Méthode	URI	Contrôleur	Description
`app_home`	GET	`/`	`HomeController::index`	Page d'accueil
`app_category`	GET	`/velos/{slug}`	`CatalogController::category`	Liste produits par catégorie

Route	Méthode	URI	Contrôleur	Description
`app_product_detail`	GET	`/produit/{slug}`	`ProductController::detail`	Fiche produit
`app_cart`	GET	`/panier`	`CartController::index`	Afficher le panier
`app_cart_add`	POST	`/panier/ajouter/{id}`	`CartController::add`	Ajouter au panier
`app_cart_update`	POST	`/panier/modifier`	`CartController::update`	Modifier quantités
`app_cart_remove`	POST	`/panier/supprimer/{id}`	`CartController::remove`	Supprimer du panier
`app_checkout`	GET/POST	`/commande`	`CheckoutController::information`	Étape 1 : Informations
`app_checkout_payment`	GET/POST	`/commande/paiement`	`CheckoutController::payment`	Étape 2 : Paiement
`app_order_confirmation`	GET	`/commande/confirmation/{ref}`	`OrderController::confirmation`	Étape 3 : Confirmation
`app_login`	GET/POST	`/connexion`	`SecurityController::login`	Connexion
`app_register`	GET/POST	`/inscription`	`SecurityController::register`	Inscription
`app_logout`	GET	`/deconnexion`	`SecurityController::logout`	Déconnexion

7.3 Exemple de contrôleur : `HomeController`

```php
<?php
// src/Controller/HomeController.php

namespace App\Controller;

use App\Repository\CategorieRepository;
use App\Repository\ProduitRepository;
use Symfony\Bundle\FrameworkBundle\Controller\AbstractController;
use Symfony\Component\HttpFoundation\Response;
use Symfony\Component\Routing\Attribute\Route;

class HomeController extends AbstractController
{
    #[Route('/', name: 'app_home')]
    public function index(
        ProduitRepository $produitRepository,
        CategorieRepository $categorieRepository
    ): Response {
        return $this->render('pages/home/index.html.twig', [
            'bestSellers' => $produitRepository->findBestSellers(8),
            'categories'  => $categorieRepository->findAll(),
        ]);
    }
}
```

7.4 Exemple de contrôleur : `CartController`

```php
<?php
// src/Controller/CartController.php
```

```php
namespace App\Controller;

use App\Entity\Produit;
use App\Service\CartService;
use Symfony\Bundle\FrameworkBundle\Controller\AbstractController;
use Symfony\Component\HttpFoundation\Request;
use Symfony\Component\HttpFoundation\Response;
use Symfony\Component\Routing\Attribute\Route;

#[Route('/panier')]
class CartController extends AbstractController
{
    #[Route('', name: 'app_cart')]
    public function index(CartService $cartService): Response
    {
        return $this->render('pages/cart/index.html.twig', [
            'cart'  => $cartService->getCart(),
            'total' => $cartService->getTotal(),
        ]);
    }

    #[Route('/ajouter/{id}', name: 'app_cart_add', methods: ['POST'])]
    public function add(Produit $produit, Request $request, CartService $cartService): Response
    {
        $taille   = $request->request->get('taille', 'M');
        $quantite = (int) $request->request->get('quantite', 1);

        $cartService->add($produit, $quantite, $taille);

        $this->addFlash('success', 'Produit ajouté au panier.');

        return $this->redirectToRoute('app_cart');
    }

    #[Route('/supprimer/{id}', name: 'app_cart_remove', methods: ['POST'])]
    public function remove(int $id, CartService $cartService): Response
    {
        $cartService->remove($id);

        return $this->redirectToRoute('app_cart');
    }
}
```

---

8. Intégration des maquettes HTML/CSS

8.1 Correspondance pages HTML → templates Twig

Fichier HTML actuel	Template Twig	Route
`index.html`	`pages/home/index.html.twig`	`app_home`
`category-road.html`	`pages/catalog/category.html.twig`	`app_category`
`product-detail.html`	`pages/catalog/product_detail.html.twig`	`app_product_detail`
`cart.html`	`pages/cart/index.html.twig`	`app_cart`
`checkout.html`	`pages/checkout/information.html.twig`	`app_checkout`
`payment-stripe.html`	`pages/checkout/payment.html.twig`	`app_checkout_payment`
`order-confirmation.html`	`pages/checkout/confirmation.html.twig`	`app_order_confirmation`

8.2 Migration des assets

```
# Copier les fichiers CSS et images dans le projet Symfony
# Depuis l'intégration HTML :
assets/css/styles.css     →  assets/styles/app.css
assets/img/*              →  assets/images/ (ou public/uploads/ pour les produits)
assets/app/script.js      →  assets/app.js (adapter pour Webpack Encore)
```

8.3 Configuration Webpack Encore (`webpack.config.js`)

```
const Encore = require('@symfony/webpack-encore');

Encore
    .setOutputPath('public/build/')
    .setPublicPath('/build')

    .addEntry('app', './assets/app.js')

    .enableSingleRuntimeChunk()
    .cleanupOutputBeforeBuild()
    .enableSourceMaps(!Encore.isProduction())
    .enableVersioning(Encore.isProduction())

    // Copier les images dans build/
    .copyFiles({
        from: './assets/images',
        to: 'images/[path][name].[hash:8].[ext]'
    })
;

module.exports = Encore.getWebpackConfig();
```

8.4 Fichier `assets/app.js`

```
// assets/app.js
import './styles/app.css';

// Importer le JS existant (adapter depuis script.js)
document.addEventListener('DOMContentLoaded', function () {
    // ... votre code JS adapté
});
```

8.5 Processus de transformation HTML → Twig

Pour chaque page HTML :

1. **Identifier** les éléments communs (navbar, footer) → déjà dans `base.html.twig`
2. **Extraire** les composants réutilisables → atomes, molécules, organismes
3. **Remplacer** les données statiques par des variables Twig (`{{ product.nom }}`)
4. **Remplacer** les liens statiques par des routes Symfony (`{{ path('app_...') }}`)
5. **Remplacer** les chemins d'images par `{{ asset('...') }}`
6. **Ajouter** les formulaires Symfony là où nécessaire

> 💡 **Conseil** : Procédez page par page, en commençant par `index.html` (la plus simple) et en terminant par `checkout.html` (la plus complexe).

---

9. Formulaires Symfony

9.1 Formulaires à créer

```
php bin/console make:form CheckoutInformationType Adresse
php bin/console make:form RegistrationType Utilisateur
php bin/console make:form ContactType
```

9.2 Exemple : formulaire de checkout

```php
<?php
// src/Form/CheckoutInformationType.php

namespace App\Form;

use App\Entity\Adresse;
use Symfony\Component\Form\AbstractType;
use Symfony\Component\Form\Extension\Core\Type\EmailType;
use Symfony\Component\Form\Extension\Core\Type\TextType;
use Symfony\Component\Form\FormBuilderInterface;
use Symfony\Component\OptionsResolver\OptionsResolver;

class CheckoutInformationType extends AbstractType
{
    public function buildForm(FormBuilderInterface $builder, array $options): void
    {
        $builder
            ->add('prenom', TextType::class, [
                'label' => 'Prénom',
                'attr'  => ['placeholder' => 'Jean'],
            ])
            ->add('nom', TextType::class, [
                'label' => 'Nom',
                'attr'  => ['placeholder' => 'Dupont'],
            ])
            ->add('rue', TextType::class, [
                'label' => 'Adresse',
                'attr'  => ['placeholder' => '12 rue du vélo'],
            ])
            ->add('codePostal', TextType::class, [
                'label' => 'Code postal',
                'attr'  => ['placeholder' => '75000'],
            ])
            ->add('ville', TextType::class, [
                'label' => 'Ville',
                'attr'  => ['placeholder' => 'Paris'],
            ])
        ;
    }

    public function configureOptions(OptionsResolver $resolver): void
    {
        $resolver->setDefaults([
            'data_class' => Adresse::class,
        ]);
    }
}
```

9.3 Thème de formulaire Bootstrap

Dans `config/packages/twig.yaml` :

```yaml
twig:
    form_themes: ['bootstrap_5_layout.html.twig']
```

Ceci applique automatiquement les classes Bootstrap 5 à tous vos formulaires Symfony.

---

10. Sécurité et authentification

10.1 Générer l'authentification

```
# Générer le formulaire de connexion
php bin/console make:security:form-login

# Générer le formulaire d'inscription
php bin/console make:registration-form
```

10.2 Configuration `security.yaml`

```yaml
# config/packages/security.yaml
security:
    password_hashers:
        Symfony\Component\Security\Core\User\PasswordAuthenticatedUserInterface: 'auto'

    providers:
        app_user_provider:
            entity:
                class: App\Entity\Utilisateur
                property: email

    firewalls:
        main:
            lazy: true
            provider: app_user_provider
            form_login:
                login_path: app_login
                check_path: app_login
                default_target_path: app_home
                enable_csrf: true
            logout:
                path: app_logout
                target: app_home
            remember_me:
                secret: '%kernel.secret%'
                lifetime: 604800  # 1 semaine

    access_control:
        - { path: ^/admin, roles: ROLE_ADMIN }
        - { path: ^/commande, roles: ROLE_USER }
        - { path: ^/profil, roles: ROLE_USER }
```

10.3 Rôles

Rôle	Accès
`ROLE_USER`	Passer commande, voir profil, historique
`ROLE_ADMIN`	Tout ROLE_USER + back-office admin

---

11. Services et logique métier

11.1 `CartService` (gestion du panier en session)

```php
<?php
// src/Service/CartService.php

namespace App\Service;

use App\Entity\Produit;
use App\Repository\ProduitRepository;
use Symfony\Component\HttpFoundation\RequestStack;

class CartService
{
    public function __construct(
        private RequestStack $requestStack,
        private ProduitRepository $produitRepository,
    ) {}

    /**
     * Retourne le panier depuis la session.
     * Format : [['product' => Produit, 'quantite' => int, 'taille' => string], ...]
     */
    public function getCart(): array
    {
        $session = $this->requestStack->getSession();
        $cart = $session->get('cart', []);
        $cartWithData = [];

        foreach ($cart as $item) {
            $product = $this->produitRepository->find($item['id']);
            if ($product) {
                $cartWithData[] = [
                    'product'  => $product,
                    'quantite' => $item['quantite'],
                    'taille'   => $item['taille'],
                    'sousTotal' => $product->getPrix() * $item['quantite'],
                ];
            }
        }

        return $cartWithData;
    }

    public function add(Produit $produit, int $quantite = 1, string $taille = 'M'): void
    {
        $session = $this->requestStack->getSession();
        $cart = $session->get('cart', []);

        // Vérifier si le produit est déjà dans le panier (même taille)
        $found = false;
        foreach ($cart as &$item) {
            if ($item['id'] === $produit->getId() && $item['taille'] === $taille) {
                $item['quantite'] += $quantite;
                $found = true;
                break;
            }
        }

        if (!$found) {
            $cart[] = [
                'id'       => $produit->getId(),
                'quantite' => $quantite,
                'taille'   => $taille,
            ];
        }

        $session->set('cart', $cart);
    }
```

```php
    public function remove(int $productId): void
    {
        $session = $this->requestStack->getSession();
        $cart = $session->get('cart', []);

        $cart = array_filter($cart, fn($item) => $item['id'] !== $productId);

        $session->set('cart', array_values($cart));
    }

    public function getTotal(): float
    {
        return array_sum(array_map(fn($item) => $item['sousTotal'], $this->getCart()));
    }

    public function getCount(): int
    {
        $session = $this->requestStack->getSession();
        $cart = $session->get('cart', []);
        return array_sum(array_column($cart, 'quantite'));
    }

    public function clear(): void
    {
        $this->requestStack->getSession()->remove('cart');
    }
}
```

11.2 Injection du compteur panier dans toutes les pages

Créer un **Event Subscriber** ou un **Twig Global** :

```yaml
# config/services.yaml
services:
    # ... (configuration existante)

    App\Twig\CartExtension:
        tags: ['twig.extension']
```

```php
<?php
// src/Twig/CartExtension.php

namespace App\Twig;

use App\Service\CartService;
use Twig\Extension\AbstractExtension;
use Twig\Extension\GlobalsInterface;

class CartExtension extends AbstractExtension implements GlobalsInterface
{
    public function __construct(private CartService $cartService) {}

    public function getGlobals(): array
    {
        return [
            'cart_count' => $this->cartService->getCount(),
        ];
    }
}
```

11.3 `OrderService` (création de commande)

```php
<?php
// src/Service/OrderService.php

namespace App\Service;

use App\Entity\ArticleCommande;
use App\Entity\Commande;
use App\Entity\Adresse;
use App\Entity\Utilisateur;
use Doctrine\ORM\EntityManagerInterface;

class OrderService
{
    public function __construct(
        private EntityManagerInterface $em,
        private CartService $cartService,
    ) {}

    public function createOrder(Utilisateur $user, Adresse $adresse): Commande
    {
        $cart = $this->cartService->getCart();

        $commande = new Commande();
        $commande->setUtilisateur($user);
        $commande->setAdresseLivraison($adresse);
        $commande->setReference($this->generateReference());
        $commande->setStatut('en_attente');

        $totalHt = 0;

        foreach ($cart as $item) {
            $article = new ArticleCommande();
            $article->setProduit($item['product']);
            $article->setQuantite($item['quantite']);
            $article->setPrixUnitaire($item['product']->getPrix());
            $article->setTaille($item['taille']);
            $article->setCommande($commande);

            $totalHt += $item['sousTotal'];

            $this->em->persist($article);
        }

        $commande->setTotalHt($totalHt);
        $commande->setTotalTtc($totalHt * 1.20); // TVA 20%

        $this->em->persist($commande);
        $this->em->flush();

        // Vider le panier
        $this->cartService->clear();

        return $commande;
    }

    private function generateReference(): string
    {
        return date('Y') . '-' . str_pad(random_int(1, 999999), 6, '0', STR_PAD_LEFT);
    }
}
```

12. Tests

12.1 Installation

```
composer require --dev phpunit/phpunit symfony/test-pack
```

12.2 Tests unitaires

```php
<?php
// tests/Service/CartServiceTest.php

namespace App\Tests\Service;

use App\Service\CartService;
use Symfony\Bundle\FrameworkBundle\Test\KernelTestCase;

class CartServiceTest extends KernelTestCase
{
    public function testCartStartsEmpty(): void
    {
        self::bootKernel();
        $cartService = self::getContainer()->get(CartService::class);

        $this->assertEmpty($cartService->getCart());
        $this->assertEquals(0, $cartService->getTotal());
    }
}
```

12.3 Tests fonctionnels (pages)

```php
<?php
// tests/Controller/HomeControllerTest.php

namespace App\Tests\Controller;

use Symfony\Bundle\FrameworkBundle\Test\WebTestCase;

class HomeControllerTest extends WebTestCase
{
    public function testHomepageIsAccessible(): void
    {
        $client = static::createClient();
        $client->request('GET', '/');

        $this->assertResponseIsSuccessful();
        $this->assertSelectorTextContains('h1', 'Nouvelle collection');
    }

    public function testHomepageDisplaysProducts(): void
    {
        $client = static::createClient();
        $crawler = $client->request('GET', '/');

        $this->assertGreaterThan(0, $crawler->filter('.product-card')->count());
    }
}
```

12.4 Lancer les tests

```
# Tous les tests
php bin/phpunit

# Un fichier spécifique
php bin/phpunit tests/Controller/HomeControllerTest.php

# Avec couverture de code
php bin/phpunit --coverage-html var/coverage
```

---

13. Déploiement et configuration Apache

13.1 VirtualHost Apache (développement local)

```
# httpd-vhosts.conf (ou via Laragon : clic droit > Apache > sites-enabled)

<VirtualHost *:80>
    ServerName bikex.local
    DocumentRoot "C:/chemin/vers/bikex/public"

    <Directory "C:/chemin/vers/bikex/public">
        AllowOverride All
        Require all granted

        # Activer le rewriting pour Symfony
        FallbackResource /index.php
    </Directory>

    # Logs
    ErrorLog "logs/bikex-error.log"
    CustomLog "logs/bikex-access.log" combined
</VirtualHost>
```

13.2 Fichier `public/.htaccess`

```
# public/.htaccess
<IfModule mod_rewrite.c>
    RewriteEngine On
    RewriteCond %{REQUEST_URI}::$0 ^(/.+)/(.*)::\2$
    RewriteRule .* - [E=BASE:%1]
    RewriteCond %{HTTP:Authorization} .+
    RewriteRule ^ - [E=HTTP_AUTHORIZATION:%0]
    RewriteCond %{ENV:REDIRECT_STATUS} =""
    RewriteRule ^index\.php(?:/(.*)|$) %{ENV:BASE}/$1 [R=301,L]
    RewriteCond %{REQUEST_FILENAME} !-f
    RewriteRule ^ %{ENV:BASE}/index.php [L]
</IfModule>

<IfModule !mod_rewrite.c>
    <IfModule mod_alias.c>
        RedirectMatch 307 ^/$ /index.php/
    </IfModule>
</IfModule>
```

> 💡 Installez le pack Apache avec : `composer require symfony/apache-pack`

13.3 Fichier hosts (Windows)

Ajoutez dans `C:\Windows\System32\drivers\etc\hosts` :

```
127.0.0.1   bikex.local
```

13.4 Variables d'environnement (production)

```
# .env.local (production)
APP_ENV=prod
APP_SECRET=votre_secret_ici
DATABASE_URL="mysql://user:password@127.0.0.1:3306/bikex_prod?serverVersion=8.0.30&charset=utf8mb4"
```

13.5 Commandes de déploiement

```
# 1. Installer les dépendances (sans dev)
composer install --no-dev --optimize-autoloader

# 2. Compiler les assets
npm run build

# 3. Vider le cache
php bin/console cache:clear --env=prod

# 4. Exécuter les migrations
php bin/console doctrine:migrations:migrate --no-interaction

# 5. Réchauffer le cache
php bin/console cache:warmup --env=prod
```

---

14. Checklist de validation V1

14.1 Environnement

- ☐ PHP 8.3.12 installé et vérifié
- ☐ MySQL 8.0.30 fonctionnel
- ☐ Apache configuré avec VirtualHost
- ☐ Symfony CLI opérationnel
- ☐ Composer et npm installés

14.2 Base de données

- ☐ Toutes les entités créées (Categorie, Produit, Utilisateur, Commande, ArticleCommande, Adresse)
- ☐ Relations correctement configurées
- ☐ Migrations générées et exécutées sans erreur
- ☐ Fixtures opérationnelles et chargées

14.3 Templates Twig (Atomic Design)

- ☐ `base.html.twig` fonctionnel avec navbar et footer
- ☐ Atomes créés (button, price, input, image, icon)
- ☐ Molécules créées (product-card, cart-item, form-group)
- ☐ Organismes créés (navbar, footer, hero-section, product-grid)
- ☐ Toutes les pages implémentées et identiques aux maquettes HTML

14.4 Fonctionnalités

- ☐ **Page d'accueil** : hero + meilleures ventes affichées dynamiquement
- ☐ **Catalogue** : produits filtrés par catégorie, sidebar filtres
- ☐ **Fiche produit** : détails, sélection taille, ajout panier

- ☐ **Panier** : ajout, modification quantité, suppression, calcul total
- ☐ **Checkout** : formulaire informations + indicateur d'étapes
- ☐ **Paiement** : formulaire carte (maquette Stripe)
- ☐ **Confirmation** : récapitulatif commande, numéro de référence
- ☐ **Connexion/Inscription** : formulaires fonctionnels
- ☐ **Déconnexion** : redirection vers l'accueil

14.5 Qualité

- ☐ Aucune erreur PHP dans la barre de debug Symfony
- ☐ Toutes les routes fonctionnent (tester avec `php bin/console debug:router`)
- ☐ CSS identique aux maquettes HTML originales
- ☐ Responsive vérifié (mobile, tablette, desktop)
- ☐ Messages flash fonctionnels (succès, erreur)
- ☐ Code versionné sur Git avec commits clairs

14.6 Sécurité (V1)

- ☐ Mots de passe hachés (bcrypt/argon2)
- ☐ Protection CSRF activée sur les formulaires
- ☐ Access control configuré (`/commande` = `ROLE_USER`, `/admin` = `ROLE_ADMIN`)
- ☐ Variables sensibles dans `.env.local` (non commité)

Commandes mémo rapide

```
# ─── DÉVELOPPEMENT QUOTIDIEN ───
symfony server:start          # Lancer le serveur
php bin/console debug:router  # Voir toutes les routes
php bin/console cache:clear   # Vider le cache

# ─── BASE DE DONNÉES ───
php bin/console make:entity           # Créer/modifier une entité
php bin/console make:migration        # Générer une migration
php bin/console d:m:m                 # Exécuter les migrations
php bin/console d:f:l                 # Charger les fixtures

# ─── GÉNÉRATION DE CODE ───
php bin/console make:controller       # Nouveau contrôleur
php bin/console make:form             # Nouveau formulaire
php bin/console make:twig-extension   # Extension Twig

# ─── ASSETS ───
npm run dev                  # Compiler les assets (développement)
npm run watch                # Compiler en continu (watch mode)
npm run build                # Compiler pour la production

# ─── TESTS ───
php bin/phpunit              # Lancer tous les tests

# ─── DEBUG ───
php bin/console debug:container     # Services disponibles
php bin/console debug:config        # Configuration des bundles
php bin/console doctrine:schema:validate  # Valider le schéma BDD
```

> 📄 **Export PDF** : `pandoc guide-execution-bikex.md -o guide-execution-bikex.pdf --toc -V geometry:margin=2cm`

Ce guide est un document de référence pour l'exécution du projet BikeX V1. Il doit être suivi séquentiellement, de la section 1 à la section 14.